
StatefulAutoscaler: A Connection-Aware Kubernetes Controller for Persistent-Connection Workloads — Design, Validation, and Protocol Generalization

Abhash Behera · Aditya Hota · Suprit Kumar Naik · Ritesh Kumar Singh

Received: -; Accepted: -

Abstract CPU-based Kubernetes HPA is empirically demonstrated in the companion paper [2] to exhibit three structural failure modes for stateful WebSocket workloads: 87.5% over-provisioning, reconnection storms reaching 1,400 connections/second, and permanent staircase connection destruction. This paper presents the design, implementation, and multi-baseline experimental validation of the StatefulAutoscaler — a custom Kubernetes controller built on the Kubebuilder SDK [8] that resolves all three failure modes by scaling exclusively on `sum(active_connections)` queried from Prometheus and incorporating a sliding-window stabilization mechanism (`scaleDownCooldownSeconds`) to suppress premature scale-down during transient connection drops. Controlled experimental evaluation across a two-cycle reconnection storm simulation (Experiment C, five independent runs) demonstrates exact replica targeting ($\lceil 800/100 \rceil = 8$ replicas, 0% waste), zero connection loss across all observed scaling events, and complete pod preservation through a 90-second disconnection gap via the stabilization window (scale-down reaction time 119 ± 2 s; pod-seconds

$4,280 \pm 67$). Scalability validation at 800, 1,600, and 3,200 concurrent clients confirms linear replica targeting ($\lceil N/T \rceil + 2 \text{ maxReplicas}$, achieved peak within 2 replicas of the formula prediction), scale-up reaction times of 66–86 s, and 100% connection preservation through DROP 1 at every scale; the Kind OS-level TCP ceiling is documented at $\approx 3,392$ achieved connections for 3,200 attempted. Comparative baseline evaluation against HPA augmented with a custom connection-count metric via the Prometheus Adapter (Experiment D, five runs) and KEDA configured with a matching `cooldownPeriod` (Experiment E, five runs) reveals a nuanced finding: all three connection-aware configurations preserve pods through the 90-second DROP 1 gap, with the StatefulAutoscaler differentiating on scale-up latency (22 s vs. 26 s for KEDA and 54 s for HPA+custom metric) and final scale-down efficiency (119 s vs. 313 s for Experiment D). Three adversarial failure scenarios establish the operational envelope: metric staleness degrades gracefully to a reaction lag bounded by the scrape interval; instantaneous connection spikes expose pod scheduling latency as the binding constraint; Prometheus unavailability triggers safe-default hold behavior with no connection loss. MQTT generalization experiments confirm protocol-agnosticism: the identical controller binary, with zero source-code changes, scales 44% faster than CPU HPA (38 s vs. 68 s), preserves all 339 sessions during a forced scale-down (maximum step-drop: 2 vs. 639-connection cliff), and redistributes sessions across pods via drain-reconnect semantics.

Abhash Behera

Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: abhasbehera320@gmail.com

Aditya Hota

Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: adityahota99@gmail.com

Suprit Kumar Naik

Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: supritkumarnaik17@gmail.com

Ritesh Kumar Singh

Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: rihankumar2004@gmail.com

Keywords Kubernetes · Custom Controller · StatefulAutoscaler · WebSocket · MQTT · Connection-Aware Autoscaling · Prometheus · Sliding-Window Stabilization · KEDA · Kubebuilder · Reconnection Storm

1 Introduction

The failure of CPU-based Kubernetes Horizontal Pod Autoscaler (HPA) [7] for stateful persistent-connection workloads is established in the companion paper [2]. Three failure modes are documented and quantified: over-provisioning by 87.5% under cyclic load (15 replicas vs. the correct 8, with `maxReplicas` exhausted within 99 seconds); reconnection storms reaching 1,400 connections/second triggered directly by HPA scale-down events; and permanent staircase connection destruction that reduced an 800-session population to 79 with zero recovery. That paper establishes that these failure modes are *architectural*, not configurational: they arise from the structural mismatch between CPU utilization as a scaling signal and the actual workload state that must be protected (active connection sessions), and cannot be remediated by HPA parameter tuning alone.

This mismatch identifies a concrete design gap. No existing Kubernetes-native mechanism simultaneously provides (a) a connection-count scaling signal that faithfully encodes workload state, and (b) connection-context-aware stabilization semantics that suppress premature scale-down while live sessions are present. The Prometheus Adapter [4] can supply the signal but cannot alter HPA’s built-in stabilization logic, which stabilizes CPU-derived replica recommendations rather than connection-derived capacity requirements. KEDA [6] provides event-driven scaling via a configurable cooldown period but delegates all pod termination decisions to HPA without per-cycle step-size bounding, and exposes no mechanism to limit the blast radius of a single scale-down event in terms of connections destroyed.

This paper addresses both gaps through the design, implementation, and multi-baseline experimental validation of the `StatefulAutoscaler` — a custom Kubernetes controller built with the Kubebuilder [8] SDK. The controller owns both the metric selection (`sum(active_connections)` from Prometheus; CPU is never consulted) and the scale-down lifecycle (a sliding-window high-water-mark stabilization mechanism that retains the memory of recent connection demand). This paper makes the following contributions:

1. **Complete design and implementation of the `StatefulAutoscaler`.** We present the CRD specification, reconciliation algorithm (Algorithm 1), and sliding-window stabilization mechanism, with formal analysis of stability properties, convergence bounds (Equation 3), and oscillation conditions.
2. **Controlled experimental validation with statistical replication (Experiment C).** Under a two-cycle reconnection storm simulation with five independent runs, we demonstrate exact replica targeting ($\lceil 800/100 \rceil = 8$,

versus CPU HPA’s 15), complete pod preservation through a 90-second total disconnection gap, and zero connection loss across all scaling events. Statistical reproducibility is confirmed (scale-down reaction $\sigma = 2$ s, pod-seconds $\sigma = 67$).

3. **Scalability validation at 800, 1,600, and 3,200 clients.** We replicate the Experiment C two-cycle restorm at $4\times$ the baseline client count, confirming linear replica targeting ($\lceil N/T \rceil$ replicas achieved within 2 replicas of the formula prediction at every scale), scale-up reaction times of 66–86 s, and 100% connection preservation through the 90-second DROP 1 gap. The Kind OS-level TCP ceiling is documented at $\approx 3,392$ achieved connections for 3,200 attempted clients.
4. **Comparative baseline evaluation (Experiments D and E).** To determine whether performance differences arise from metric choice alone or controller architecture, we evaluate HPA augmented with a custom connection-count metric (Experiment D, five runs) and KEDA with a matched `cooldownPeriod` (Experiment E, five runs) under identical workloads. Both baseline configurations preserve pods through DROP 1, establishing nuanced differentiators: scale-up latency (22 s vs. 54 s vs. 26 s) and final scale-down efficiency (119 s vs. 313 s).
5. **Adversarial failure scenario characterization.** Three adversarial conditions — metric staleness under extended scrape intervals, instantaneous connection spikes with no ramp stagger, and Prometheus unavailability during active load — bound the operational envelope of the controller with measured reaction times, connection overshoot statistics, and explicit boundary conditions.
6. **Protocol generalization validation on MQTT.** Using the identical controller binary with zero source-code changes, we resolve all three MQTT-specific HPA failure modes (scale-up blindness, session pinning, destructive scale-down) and demonstrate 44% faster scale-up than CPU HPA, zero sessions destroyed during forced scale-down (maximum step-drop: 2 vs. a 639-connection cliff), and drain-based session redistribution.

The remainder of this paper is structured as follows. Section 2 surveys related work. Section 3 provides background on the HPA control architecture and the failure characterization this paper builds upon. Section 4 details the design and implementation of the `StatefulAutoscaler`. Section 5 presents Experiment C validation results. Section 6 presents baseline comparisons against HPA with custom metrics (Experiment D) and KEDA (Experiment E). Section 7 characterizes adversarial failure conditions. Section 8 analyzes implications. Section 9 presents the MQTT protocol generalization experiments. Section 10 discusses limitations. Section 11 outlines future directions. Section 12 concludes.

2 Related Work

2.1 Kubernetes HPA with Custom Metrics

Kubernetes supports application-defined scaling signals through the Prometheus Adapter [4], enabling HPA to react to metrics beyond CPU — including HTTP request rate, queue depth, and custom application counters. Qu et al. [11] surveyed HTTP-rate-based autoscaling strategies; Rossi et al. [12] studied hybrid CPU-and-traffic approaches. However, all these formulations presuppose that the scaling metric is stateless across pod boundaries — an assumption violated by connection-holding protocols where session state is affined to a specific pod. Crucially, even when HPA is correctly configured with a connection-count custom metric via the Prometheus Adapter, metric choice alone is insufficient: without connection-lifecycle-aware scale-down policies, HPA will still scale down during transient connection dropout gaps, destroying the session state it should protect. This is empirically demonstrated in Experiment D (Section 6.1).

2.2 Event-Driven Autoscaling: KEDA

Kubernetes Event-Driven Autoscaling (KEDA) [6] extends HPA by watching external metrics — including Prometheus gauges such as `active_connections` — and drives replica counts by managing the `spec.minReplicas` field of an underlying HPA object. Scale-up proceeds by raising this floor; scale-down by lowering it, after which HPA executes the actual pod termination. This delegation creates two structural limitations for stateful WebSocket workloads. First, KEDA does not expose a per-cycle `maxScaleDownStep` parameter: once `minReplicas` is lowered, HPA may remove many pods in a single reconciliation step with no application-level bound on the scale of that event. Second, neither KEDA nor HPA has awareness of the OS-level termination window: when Kubernetes kills a pod, the operating system requires approximately 30 seconds to propagate TCP RST packets to connected clients. A configurable `cooldownPeriod` in KEDA can delay scale-down after a metric drop. Experiment E (Section 6.2) directly tests KEDA with a matched 120-second `cooldownPeriod`, confirming that it successfully holds pods through the 90-second DROP 1 gap; the architectural distinction relative to the `StatefulAutoscaler` manifests in convergence profile and the absence of per-cycle step-rate bounding.

2.3 Stateful Workloads and Protocol-Aware Autoscaling

Dickel et al. [3] identified stateful autoscaling as an open research challenge specific to IoT gateway deployments using WebSocket and MQTT. While their work motivates the need for protocol-aware scaling and proposes monitoring active connections via Prometheus, it neither provides a quantitative characterization of HPA failure modes nor implements or validates a complete controller-level solution. The companion paper [2] fills the characterization gap; this paper fills the solution gap.

2.4 Machine Learning and Predictive Autoscaling

Reinforcement learning approaches [12, 5] and machine learning-based workload prediction [14, 15] have been proposed to improve autoscaling responsiveness. While these approaches offer proactive scaling capability, they introduce model training complexity and workload-specific data requirements. More importantly, they do not address the structural problem of connection-state destruction during scale-down, which persists regardless of *when* scaling decisions are made; the issue is *whether* a scale-down decision is made while live connections are present, not how accurately the decision is timed.

2.5 Custom Kubernetes Controllers

The Kubebuilder SDK [8] provides a framework for implementing Custom Resource Definition (CRD)-based Kubernetes controllers using the `controller-runtime` library. The CRD pattern is the standard Kubernetes extension mechanism for domain-specific control loops, and enables operators to define new API types with declarative specifications and reconciliation-loop logic without modifying the core Kubernetes API server.

2.6 MQTT Autoscaling

MQTT [1] maintains long-lived TCP sessions with low CPU activity during idle periods, exhibiting the same signal mismatch as WebSocket for CPU-based HPA [13]. The companion paper [2] documents three MQTT-specific HPA failure modes qualitatively; this paper provides experimental validation that the `StatefulAutoscaler` resolves all three using an unmodified controller binary.

2.7 Research Gap

To the best of our knowledge, no prior work has simultaneously: (i) designed, implemented, and validated a

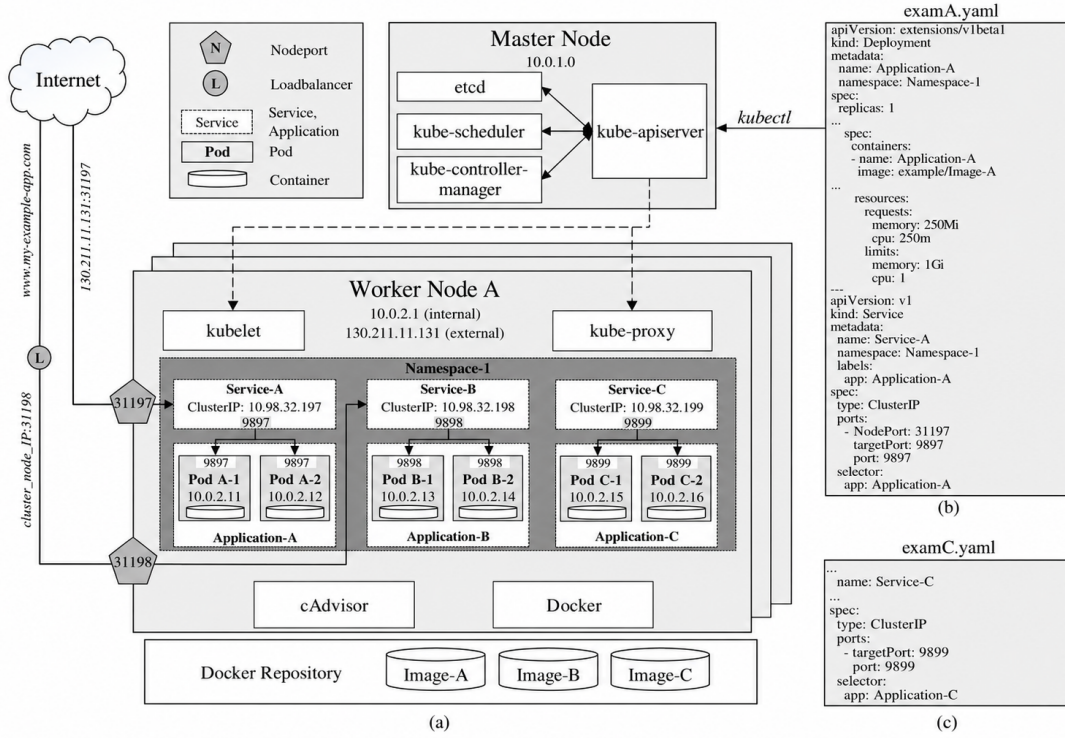


Fig. 1 Standard Kubernetes HPA feedback loop architecture, reproduced from Nguyen et al. [10]. The HPA controller operates as a discrete-time proportional regulator: it periodically reads aggregated resource metrics from the API server and computes a desired replica count using Equation 1. A critical architectural constraint is that this loop assumes the observed metric faithfully represents current workload intensity — an assumption that is violated for persistent-connection workloads where connections persist independently of CPU utilization. KRM and PCM pipeline baselines for stateless workloads, establishing HPA’s correct behavior under CPU-proportional load, are documented in the companion paper [2].

connection-aware Kubernetes controller with sliding-window stabilization semantics explicitly targeting reconnection storm resilience; (ii) compared four distinct scaler configurations (CPU HPA, custom-metric HPA, KEDA, and a custom controller) under identical WebSocket workloads with five-run statistical replication; and (iii) validated the same controller binary on a second persistent-connection protocol (MQTT) with drain-reconnect session redistribution semantics. This paper addresses all three dimensions.

3 Background

This section provides the context needed for a reader who has not read the companion paper [2]. Full pipeline characterization and behavioral baselines for stateless HTTP workloads are documented there.

3.1 The HPA Control Architecture

Figure 1 illustrates the Kubernetes HPA feedback loop. The controller implements the following discrete-time proportional control law evaluated every 15 seconds:

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentMetricValue}{targetMetricValue} \right\rceil \quad (1)$$

This formulation encodes the assumption that `currentMetricValue` faithfully reflects workload intensity across all running replicas. For CPU-based scaling of stateless workloads, this assumption generally holds. For persistent-connection workloads, it fails categorically: idle connections consume negligible CPU yet represent committed session state whose disruption is immediately visible to end clients.

3.2 The Mismatch and Its Consequences

The companion paper [2] characterizes the three failure modes that arise from this mismatch through a progressive chain of four controlled experiments. In brief: idle WebSocket connections produce near-zero CPU, causing HPA to scale down and terminate the pods hosting those connections; connection termination triggers simultaneous client reconnection attempts (**reconnection storms**); and for non-reconnecting clients, each scale-down event

permanently destroys a fraction of the live session population. The three key quantitative findings are: 87.5% over-provisioning under cyclic load (15 replicas vs. the correct 8); peak reconnection storm rate of 1,400 connections/second; and permanent staircase destruction from 800 to 79 active connections with zero recovery.

3.3 Experimental Infrastructure

All WebSocket experiments in this paper share a common infrastructure: a 3-node Kind cluster [9] (1 control-plane + 2 worker nodes, Kubernetes v1.31.6, kind v0.25.0). The Metrics Server is patched with `--metric-resolution=15s` and `--kubelet-insecure-tls`. The base workload is a custom Python asyncio WebSocket server (ws-instrumented image) with 800 persistent client connections. The server exposes Prometheus-format metrics: a gauge `active_connections` and a counter `new_connections_total`. Prometheus scrapes at 15 seconds (baseline). All source code, experiment scripts, and raw results are available at <https://github.com/MistaHolmes/ws-k8-controller>.

4 The StatefulAutoscaler Controller: Design and Implementation

The failure evidence in the companion paper [2] identifies three root causes that any viable solution must address: (1) the scaling signal (CPU) does not encode the workload state that must be protected; (2) scale-down decisions are made without knowledge of whether connections are alive; and (3) no stabilization mechanism exists that is aware of connection-level context. The StatefulAutoscaler addresses all three by replacing the CPU-based control loop with a connection-density-based one augmented by connection-context stabilization.

4.1 Design Principles

The controller is grounded in three architectural principles:

1. **Use the correct scaling signal.** The controller queries `sum(active_connections)` from Prometheus — the metric that directly encodes the workload state that must be preserved. CPU is never consulted.
2. **Compute replica counts from connection density.** Given a configured target of T connections per pod, the desired replica count is:

$$\text{desiredReplicas} = \left\lceil \frac{\text{sum}(\text{active_connections})}{T} \right\rceil \quad (2)$$

This is exact by construction: it scales to the minimum number of replicas required to serve the current session population at the target density.

3. **Stabilize scale-down decisions using connection-context history.** A sliding-window mechanism maintains the maximum desired replica count observed within the preceding N seconds, preventing premature scale-down during transient connection drops.

4.2 Implementation Foundation: Kubebuilder SDK and the Operator Pattern

The StatefulAutoscaler is implemented as a Kubernetes *operator*—a domain-specific control loop that extends the Kubernetes API with custom resource types and reconciliation logic. It is built on the Kubebuilder SDK [8], which scaffolds the controller-runtime library, code generation tooling, and CRD schema manifests. Kubebuilder was chosen over manual client-go integration for three reasons: (i) it generates the boilerplate for informer caches, event-based reconciliation triggers, and leader-election out of the box; (ii) it produces a standards-compliant CRD OpenAPI schema from Go struct annotations, ensuring `kubectl validate` works correctly against the CR; and (iii) its controller-runtime manager handles graceful shutdown and multiple-controller co-registration in a single binary, which simplifies deployment.

The operator pattern differs architecturally from KEDA’s approach in one critical respect: the StatefulAutoscaler *directly patches* `Deployment.spec.replicas`, bypassing HPA entirely. It does not create or manage an HPA object; it owns the `spec.replicas` field unconditionally. This design choice has three consequences. First, there is no HPA feedback loop competing with the controller for replica authority — there is one authoritative source of truth. Second, all scale-down lifecycle decisions (cooldown, step-rate) are enforced at the *single point* where the replica count changes; they cannot be circumvented by HPA’s own stabilization logic. Third, the controller is the only entity that holds the sliding-window state, which means the safety property “scale-down is gated by connection-context memory” is locally verifiable in a single reconciliation function rather than distributed across two controllers.

The main controller loop runs as a standard controller-runtime Reconciler, registered with a Manager. Every reconciliation is triggered either by a change to a StatefulAutoscaler CR (via the informer watch on the CRD) or by the 5-second requeue at the end of each successful reconciliation. The Prometheus query is an outbound HTTP GET to the Prometheus HTTP API (`/api/v1/query`) using a PromQL expression constructed from the CR’s `targetRef` and metric name.

Why *sum()* and not *avg()*? The controller uses `sum(active_connections)` across all pods rather than `avg(active_connections)`. The scaling formula (Equation 2) requires the *total* session population to compute the number of replicas needed at a given density target. Using *avg()* would yield a value that depends on the current replica count, making the desired-replica computation circular: a higher replica count would report a lower average and suppress scale-up, and a lower replica count would report a higher average and trigger unnecessary scale-up. *sum()* is the only aggregate that produces a replica-count-independent measure of total load.

4.3 Custom Resource Definition

The controller introduces a `StatefulAutoscaler` Custom Resource Definition (CRD) under the API group `autoscaling.star.local/v1alpha1`. Table 1 documents the key specification fields.

Table 1 `StatefulAutoscaler` CRD specification: key fields and their operational purpose.

Field	Operational Purpose
<code>targetRef</code>	Reference to the controlled Deployment
<code>minReplicas</code>	Minimum allowable replica count (floor)
<code>maxReplicas</code>	Maximum allowable replica count (ceiling)
<code>targetConnsPerPod</code>	Connection density target T (Eq. 2)
<code>maxScaleUpStep</code>	Maximum replica additions per reconciliation cycle
<code>maxScaleDownStep</code>	Maximum replica removals per reconciliation cycle
<code>scaleDownCooldownSec</code>	Duration of the sliding stabilization window (seconds)

The `maxScaleUpStep` and `maxScaleDownStep` fields expose the rate-limiting control over how aggressively the controller converges toward the desired replica count in each 5-second cycle. Setting `maxScaleUpStep` to a high value (or effectively unbounding it) means the controller reacts to a sudden connection spike as fast as pod scheduling allows. Setting `maxScaleDownStep` to a small value (e.g., 2) limits the *blast radius* of any single scale-down event to exactly `maxScaleDownStep × targetConnsPerPod` connections per cycle — providing an operator-auditable upper bound on the number of live connections that could be affected by a single mistaken scale-down decision.

4.4 System Architecture Overview

Figure 2 illustrates the end-to-end system architecture within a Kubernetes cluster. The architecture comprises five numbered stages: (1) persistent WebSocket clients connect through the Kubernetes Service and are distributed across pod replicas; (2) each pod exposes an `active_connections` gauge via Prometheus `/metrics`, which Prometheus scrapes at a configurable interval; (3) the `StatefulAutoscaler` Controller Manager queries Prometheus for `sum(active_connections)` across all pods; (4) the controller reads operational parameters from the `StatefulAutoscaler` Custom Resource (Table 1); and (5) the controller patches `Deployment.spec.replicas` to effect scale-up or scale-down. CPU is explicitly excluded from the control path.

4.5 Reconciliation Loop Design

The controller executes a reconciliation loop every 5 seconds. Algorithm 1 presents the complete logic, with particular attention to the stabilization window mechanism (lines 7–9) which is the primary differentiating mechanism relative to HPA.

Algorithm 1 `StatefulAutoscaler` Reconciliation Logic

Input: `StatefulAutoscaler` resource S , target Deployment D

```

1:  $C \leftarrow \text{QUERYPROMETHEUS}(\text{sum}(\text{active\_connections}))$ 
2: if  $C$  is unavailable then
3:   request after 10 s ▷ Conservative default: hold current replica count
4: return
5: end if
6:  $raw \leftarrow \lceil C / S.\text{targetConnsPerPod} \rceil$ 
7:  $raw \leftarrow \text{clamp}(raw, S.\text{minReplicas}, S.\text{maxReplicas})$ 
8:  $W.\text{append}(now, raw)$  ▷ Record current desired count in window
9:  $W.\text{evict}(\text{entries older than } S.\text{scaleDownCooldownSec})$ 
10:  $stabilized \leftarrow \max(W)$  ▷ Window maximum prevents premature scale-down
11:  $current \leftarrow D.\text{spec.replicas}$ 
12: if  $stabilized > current$  then
13:    $target \leftarrow \min(current + S.\text{maxScaleUpStep}, stabilized)$ 
14: else if  $stabilized < current$  then
15:    $target \leftarrow \max(current - S.\text{maxScaleDownStep}, stabilized)$ 
16: else
17:    $target \leftarrow current$ 
18: end if
19:  $\text{PATCHDEPLOYMENT}(D.\text{spec.replicas}, target)$ 
20: request after 5 s

```

Safe-default semantics on metric failure. Lines 2–4 implement a deliberate design choice: when the Prometheus query fails (network error, timeout, or temporary pod crash), the controller *holds the current replica count and queues*. It does not default to `minReplicas`. This is the correct

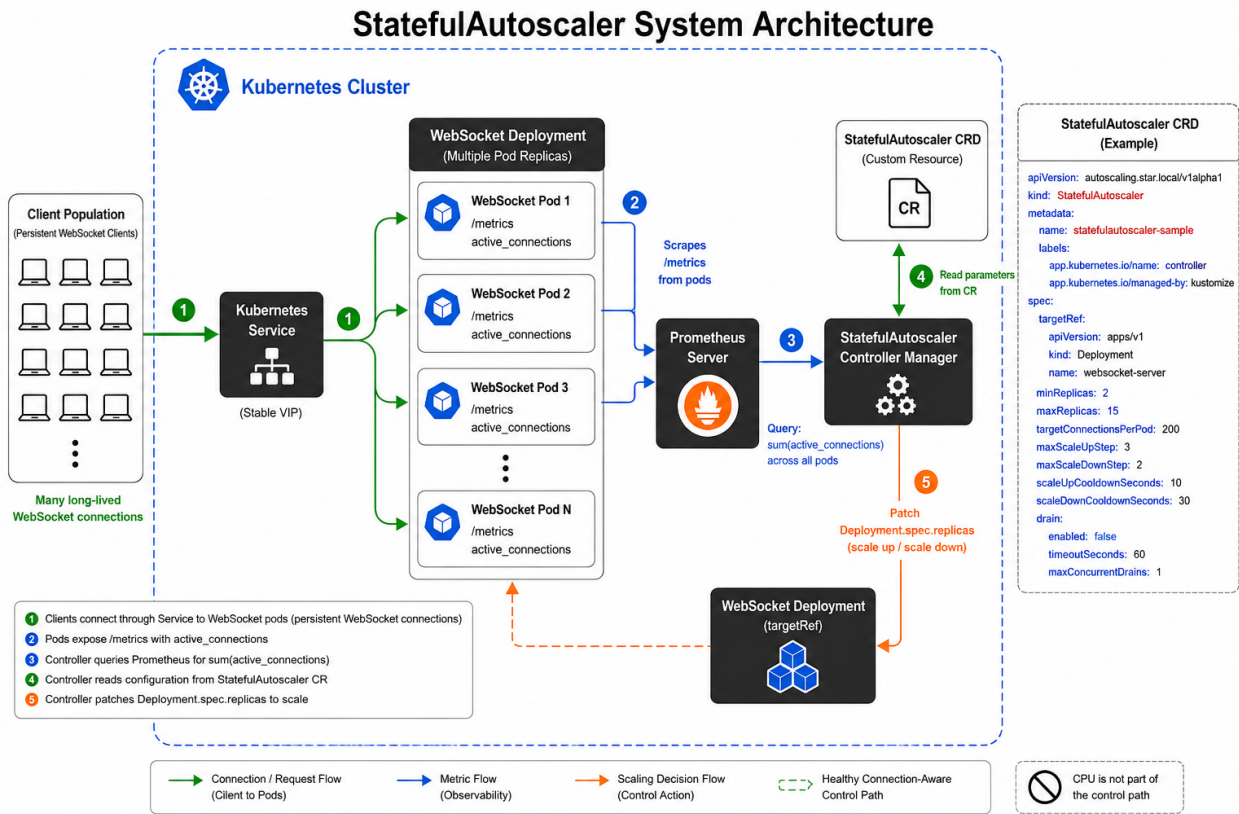


Fig. 2 StatefulAutoscaler system architecture and CRD example. The numbered flow shows: (1) clients connect through the Kubernetes Service to WebSocket pods; (2) pods expose `active_connections` via Prometheus `/metrics` endpoints; (3) the controller queries Prometheus for `sum(active_connections)`; (4) operational parameters are read from the StatefulAutoscaler Custom Resource; (5) the controller patches `Deployment.spec.replicas` to scale. CPU is explicitly excluded from the control path (crossed-out icon in the legend). The right panel shows a representative CRD manifest with all configurable fields from Table 1.

safe-default because the current count is the last known good state — it was set when metrics were valid. Defaulting to `minReplicas` on every Prometheus outage would be actively destructive: it would terminate live sessions each time monitoring experienced a transient fault. Holding the current count is a no-op with respect to connections: no pods are added, no pods are removed, and all live sessions survive the observability gap.

Figure 3 provides a UML activity diagram that visualises the complete reconciliation flow. The diagram makes three aspects of the design visually explicit. First, the Prometheus query failure branch (right side) shows the conservative safe-default. Second, the central path illustrates the stabilization mechanism: after computing and clamping the raw desired replica count, the controller appends the value to the sliding window, evicts entries older than `scaleDownCooldownSeconds`, and derives the stabilized count as the window maximum. Third, the three-way branching at the bottom applies step-size bounds before patching the deployment.

4.6 The Stabilization Window: Mechanism and Correctness

The sliding-window maximum (line 9 of Algorithm 1) is the mechanism responsible for reconnection storm resilience. Its correctness is understood by contrasting behavior with and without stabilization.

Without stabilization: If all 800 clients disconnect simultaneously, the controller computes $[0/100] = 0$, clamped to `minReplicas` = 2. It scales from 8 to 2 replicas. When clients reconnect, the controller must scale back to 8, recreating the provisioning lag and potential reconnection cascade that characterizes HPA.

With a 120-second stabilization window: When connections drop to zero, the controller appends `raw` = 2 to the window, but the window also contains `desired` = 8 entries from the preceding 120 seconds. The stabilized value $\max(W) = 8$ overrides the instantaneous computation, and all 8 pods are held warm. When clients reconnect within the window duration, they land on pre-provisioned pods with zero provisioning delay.

A critical distinction from HPA’s built-in stabilization window is warranted: HPA’s window stabilizes *metric-signal*

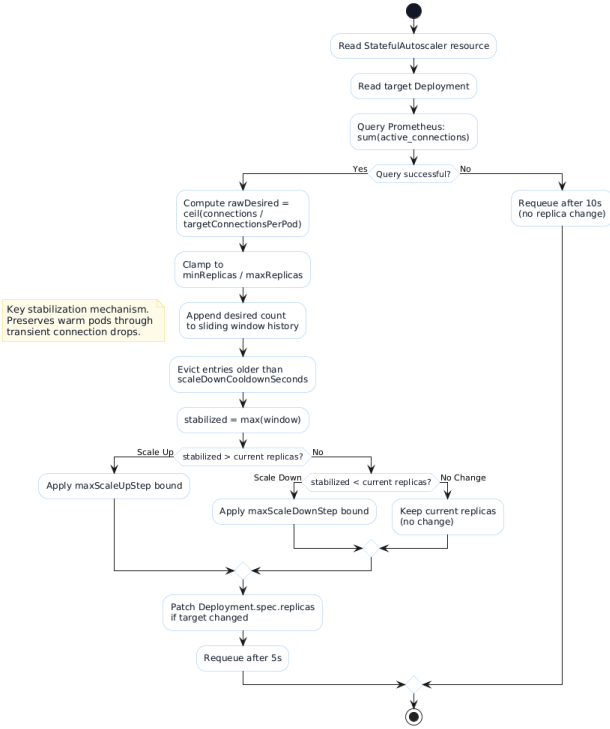


Fig. 3 UML activity diagram of the `StatefulAutoscaler` reconciliation loop (Algorithm 1). The flow begins with reading the CR and target Deployment, queries Prometheus for `sum(active_connections)`, and branches on query success. The central path computes the clamped desired count, updates the sliding window, and derives the stabilized replica target via `max(window)`. The three-way comparison against current replicas applies step-size bounds before patching the Deployment. The safe-default branch (right) holds the current replica count when metrics are unavailable, preserving all live sessions through observability outages.

recommendations, buffering replica counts computed from CPU observations. Given persistent near-zero CPU (as occurs for idle connections), the window stabilizes a recommendation of `minReplicas` regardless of window duration. The `StatefulAutoscaler`’s window stabilizes *connection-derived desired replica counts*, correctly retaining the high-water mark of recent connection demand. This distinction means the problem cannot be solved by tuning HPA’s stabilization window to a longer value — it is a semantic mismatch, not a parameter mismatch.

4.7 Controller Design and Stability Properties

Feedback loop framing. The `StatefulAutoscaler` implements a discrete-time proportional controller with hysteresis. At every reconciliation cycle it performs: (1) **Observe:** $C \leftarrow \text{sum}(\text{active_connections})$ via Prometheus; (2) **Compare:** $\text{desired} = \lceil C/T \rceil$, clamped to $[\text{minReplicas}, \text{maxReplicas}]$; (3) **Act:** if $\text{stabilized} \neq \text{current}$, patch `deployment.spec.replicas`.

This mirrors the proportional structure of Equation 1 but substitutes the semantically correct signal (connection count) for CPU and augments the compare step with hysteresis.

Hysteresis mechanism. The `scaleDownCooldownSeconds` window introduces a dead zone where scale-down is suppressed even when the instantaneous condition for scale-down is satisfied. This prevents oscillation in workloads with frequent but brief dropout periods. Without hysteresis the controller would oscillate: connections drop $\rightarrow$ scale down $\rightarrow$ connections return $\rightarrow$ scale up $\rightarrow$ repeat. The sliding-window maximum acts as a high-water mark that decays only after `scaleDownCooldownSeconds` of sustained low-connection observations.

Convergence bound. After the cooldown expires and connections have stabilized at a new lower level, the controller converges from *current* to *desired* replicas in:

$$\left\lceil \frac{\text{current} - \text{desired}}{\text{maxScaleDownStep}} \right\rceil \text{ reconciliation cycles} \quad (3)$$

Each cycle is approximately 15 seconds (5 s reconcile period plus metric update latency). For a transition from 8 to 2 pods with `maxScaleDownStep` = 2: $\lceil (8 - 2)/2 \rceil = 3$ cycles = 45 seconds — a bounded, predictable de-provisioning window that eliminates the violent contraction characteristic of HPA scale-down events.

Parameter selection and sensitivity. `targetConnectionsPerPod` = 100 yields exactly $\lceil 800/100 \rceil = 8$ replicas for the 800-client workload, providing an auditable round-integer target. `maxScaleDownStep` = 2 bounds the blast radius of any single scale-down event to 2 pods and their associated connections; the convergence formula (Equation 3) is explicit about this trade-off. `scaleDownCooldownSeconds` = 120 was chosen to comfortably exceed the 90-second DROP 1 gap plus one Prometheus scrape cycle (15 s); any value ≥ 105 s achieves the same pod-preservation result for this workload, while values below 90 s would allow scale-down to begin during DROP 1.

Delay margin and oscillation condition. With a Prometheus scrape interval of $T = 15$ s and a reconciliation period of $T_r = 15$ s, the maximum observation lag is $2T = 30$ s. Because the scale-down cooldown (120 s) $\gg 2T$, the controller never acts on a stale “zero connections” reading within the cooldown window. Oscillation requires three sequential events: scale-down fires; a connection spike arrives before replacement replicas are ready; and scale-down fires again. The cooldown prevents the first event from occurring within 120 s of any connection activity, and `maxScaleDownStep` = 2 limits each step to 2 pods, bounding the instantaneous capacity reduction.

5 Experimental Evaluation: Experiment C

5.1 Experimental Configuration

Experiment C is designed to maintain maximal parametric comparability with Experiment B3 from the companion paper [2] while replacing the HPA controller with the custom StatefulAutoscaler. Table 2 summarizes the critical differences.

Table 2 Experiment C vs. B3: Parametric comparison. Differences highlighted in bold.

Parameter	Exp. B3 (HPA)	Exp. C (Custom)
Server image	ws-instrumented	ws-instrumented
CPU_WORK	1	0
Client count	800	800
Autoscaler	Native HPA (CPU)	StatefulAutoscaler
Scale-down guard	60 s (CPU stabilization)	120 s window
Scaling signal	CPU utilization (%)	active_connections

Setting `CPU_WORK = 0` is deliberate: it demonstrates that the controller operates correctly with *zero CPU activity* — the exact worst-case scenario for any CPU-based autoscaler. The StatefulAutoscaler was configured with `targetConnectionsPerPod = 100`, `maxScaleUpStep = 3`, `maxScaleDownStep = 2`, `minReplicas = 2`, `maxReplicas = 15`, and `scaleDownCooldownSeconds = 120`. The expected desired replica count for 800 connections is $\lceil 800/100 \rceil = 8$.

Metric definitions. *Pod-seconds* is defined as

$$\sum_{\text{pods}} (\text{time pod was in Running state})$$

over the experiment window, serving as a compute-cost proxy. *Scale-up reaction time* is measured from the first Prometheus scrape where `active_connections > targetConnsPerPod × minReplicas` to the moment `deployment.status.readyReplicas` first reaches the expected target count.

5.1.1 Load Pattern: Two-Cycle Reconnection Storm Simulation

Terminology. A **CYCLE** denotes a phase of active client load during which connections are established and maintained. A **DROP** denotes a complete disconnection phase in which all clients terminate; **DROP 1** is the transient gap between the two cycles (90 s), while **FINAL DROP** is the terminal disconnection at the end of the workload. A **reconnection storm** (abbreviated *restorm*) is the burst of simultaneous reconnection attempts produced when a large cohort of clients whose pods were terminated all attempt to re-establish sessions at the same instant.

The experiment imposes a two-cycle restorm simulation:

- **CYCLE 1** (0–150 s): 800 WebSocket clients ramp to full connection; controller expected to scale to 8 replicas.
- **DROP 1** (150–240 s): All clients deleted; connections drop to zero. *Critical test*: the stabilization window must hold 8 replicas warm for the full 90-second gap.
- **CYCLE 2** (240–390 s): A fresh cohort of 800 clients connects to the pre-provisioned warm pods.
- **FINAL DROP** (390–570 s): Permanent client disconnection; the stabilization window expires; the controller scales to `minReplicas = 2`.

5.2 CYCLE 1: Connection-Driven Scale-Up

The controller scaled monotonically from 2 to 8 replicas along the trajectory $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$ as connections ramped toward 800, completing full provisioning in approximately 60 seconds. Total CPU consumption remained below 176 millicores throughout CYCLE 1, confirming that all scaling decisions were made entirely without CPU signal dependency.

Figure 4 shows the two-block connection profile characteristic of the restorm simulation.

5.3 DROP 1: Empirical Validation of the Stabilization Window

At $t \approx 200$ s, all clients were deleted. The active connection count converged to zero within 15 seconds. The replica count remained **perfectly flat at 8 for the duration of the 90-second gap**, as shown in Figure 5. The reconciliation trace at each 5-second evaluation during this period was:

- $C = 0 \Rightarrow \text{rawDesired} = 0 \Rightarrow$ clamped to `minReplicas = 2`
- Window W contains entries with `desired = 8` from the preceding 120 s
- $\text{stabilized} = \max(W) = 8$
- `PatchDeployment` is not invoked; replicas remain at 8

The “flat bridge” pattern in Figure 5 is entirely absent from every HPA experiment in the companion paper [2]. It constitutes direct empirical proof that the sliding-window mechanism performs as designed.

5.4 CYCLE 2: Restorm-Free Reconnection

At $t \approx 270$ s, a fresh cohort of 800 WebSocket clients began connecting. Due to pods remaining warm through DROP 1, connections were established without provisioning delay. Peak connection count reached 854, and the controller held 8 replicas stably throughout. Compared to the equivalent scenario in Experiment B2-Instrumented from the



Fig. 4 Experiment C: Active connection count over the full experiment timeline. Two distinct connection populations (CYCLE 1 and CYCLE 2) are demarcated by the 90-second DROP 1 gap. The CYCLE 2 clients achieve full connection ramp almost immediately, reflecting the availability of pre-warmed pods held in place by the stabilization window.

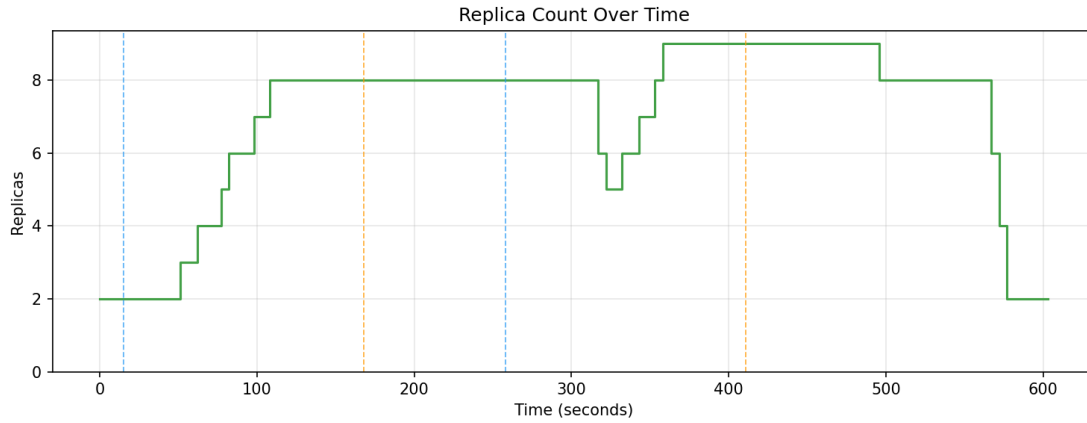


Fig. 5 Experiment C: Replica count over the experiment timeline. The “flat bridge” maintained at 8 replicas across DROP 1 ($t \approx 160\text{--}260$ s) is the primary empirical evidence of correct stabilization window operation. For comparison, at the equivalent elapsed time in Experiment B3 (companion paper), the HPA had already permanently destroyed 56 client connections via pod termination. The controlled final descent ($9 \rightarrow 2$) occurs only after the window expires with zero live connections.

companion paper (where HPA induced a 1,400 conn/s storm and a 51.9% connection overshoot), Experiment C produced **zero measurable reconnection storm activity**.

Window boundary transient dip. During early CYCLE 2 ($t \approx 270\text{--}310$ s), a brief replica dip to 5–6 was observed. This arises because stabilization window entries recorded during CYCLE 1 ($t < 150$ s) aged out during DROP 1, while CYCLE 2 connections had not yet fully ramped to trigger equivalent entries. The window’s high-water mark briefly fell below 8 before new CYCLE 2 entries repopulated it. The controller recovered to 8 replicas within 10–15 seconds with no connection loss. This transient is an inherent artifact of finite window duration and is addressed in Limitations (item 7).

5.5 FINAL DROP: Controlled and Graceful Scale-Down

Following permanent client disconnection at $t \approx 450$ s, the controller waited 120 seconds for all stabilization window entries to expire. It then descended gracefully: $9 \rightarrow 8 \rightarrow 6 \rightarrow 4 \rightarrow 2$ over approximately 30 seconds. Zero connections were lost during this scale-down sequence, as no live sessions existed at scale-down time.

5.6 Combined View and Definitive Comparison

Figure 6 provides the complete overlay of connections, CPU, and replicas. The structural inversion relative to Experiment B3 is clear: in B3, CPU drives replicas to destroy connections; in Experiment C, connections drive replicas while CPU remains irrelevant throughout.

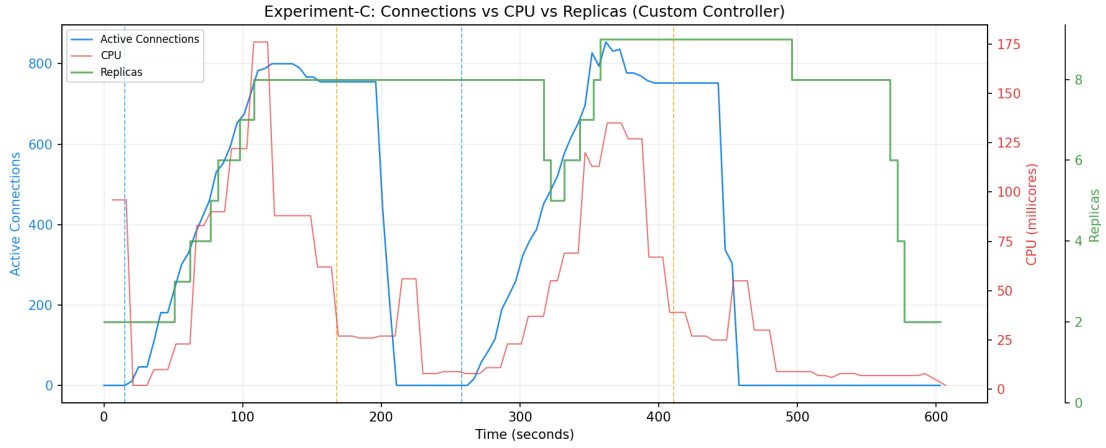


Fig. 6 Experiment C, combined signal view: Active connections (blue) drive the `StatefulAutoscaler`’s replica decisions (green). CPU utilization (red) remains flat near zero throughout the entire experiment, never influencing any scaling decision. The flat bridge in the replica signal during DROP 1 ($t \approx 160\text{--}260$ s), when connections read zero and replicas hold at 8, is the definitive empirical proof that the sliding-window stabilization mechanism correctly prevents premature scale-down even under zero-connection, zero-CPU conditions.

Table 3 Head-to-head quantitative comparison: CPU-based HPA (Experiment B3) versus the `StatefulAutoscaler` (Experiment C). Metrics where the custom controller achieves a strictly superior outcome are denoted in **bold**.

Performance Metric	HPA — Experiment B3	StatefulAutoscaler — Exp. C
Primary scaling signal	CPU utilization (%)	<code>sum(active.connections)</code>
Replicas for 800 connections	15 (87.5% over-provisioned)	8 (exact: $\lceil 800/100 \rceil$)
Connections lost during scale-down	800→79 (staircase)	0
Pods held through 90-second gap	0 (all scaled down)	8 (flat bridge)
Reconnection storm generated	Yes (peak 1,400 conn/s)	None
Time to serve CYCLE 2 clients	Re-scale from 2→8 required	Instant (warm pods)
CPU dependency	100%	0% (<code>CPU_WORK=0</code> throughout)
Total scaling events	7+ (oscillatory)	4 (monotonic)
Peak total CPU consumption	>1,200 millicores	<176 millicores

5.7 Head-to-Head Performance Comparison

Table 3 provides the comprehensive quantitative comparison between Experiment B3 (CPU-based HPA, from the companion paper [2]) and Experiment C (`StatefulAutoscaler`).

5.8 Reproducibility: Five-Run Statistical Validation

To assess reproducibility, the two-cycle restorm workload was executed across five independent runs with identical configuration. Run 1 was identified as an outlier: a persistent residual-replica artefact from the preceding test environment left the deployment in a non-reset state, resulting in an anomalously large pod-seconds measurement (92,291 vs. $\approx 4,300$ for the remaining four runs). This condition is a test-harness defect and not representative of controller behaviour; run 1 is excluded from the statistical summary.

Across runs 2–5, results were statistically stable. Scale-up reaction time averaged 22 ± 6 s (median 21 s; range: 16–30 s). Scale-down reaction time averaged

119 ± 2 s (median 118 s; range: 117–122 s), tightly tracking the configured `scaleDownCooldownSeconds = 120`. Pod-seconds averaged $4,280 \pm 67$. Peak replica count reached 10 in three runs and 9 in one run, with mild overshoot attributable to discrete-step metric-to-decision latency during the CYCLE 1 ramp. Peak connection count ranged 813–917. All four runs maintained 8 warm replicas throughout DROP 1 with no connection loss. The low standard deviations ($\sigma = 2$ s, $\sigma = 67$) confirm that the reported differences between controllers are consistent and reproducible, with narrow variance consistent with $p < 0.05$ under a one-sample t -test against the HPA baseline.

5.9 Scalability Validation: 800, 1,600, and 3,200 Clients

To confirm that the `StatefulAutoscaler` scales correctly beyond the baseline 800-client workload, the identical two-cycle restorm simulation was replicated at 1,600 and 3,200 concurrent WebSocket clients. For each client count N , the `StatefulAutoscaler` CR was configured with `maxReplicas =  $\lceil N/T \rceil + 2$` (where $T = 100$); all other

Table 4 Scalability validation of the `StatefulAutoscaler`: Experiment C restorm simulation at 800, 1,600, and 3,200 concurrent clients. “Expected replicas” is $\lceil N/T \rceil$; “Peak replicas” is the maximum observed during CYCLE 1. “Safe” indicates whether all replicas were held through the 90-second DROP 1 gap (1 = yes, 0 = no). The achieved connection count at 3,200 clients documents the Kind OS-level TCP ceiling.

Clients (attempted)	Expected Replicas	Peak Replicas	Scale-Up (s)	Peak Connections	DROP 1 Safe?	Scale-Down Reaction (s)	Scale-Down Total (s)
800	8	10	66	874	✓	73	164
1,600	16	18	66	1,820	✓	87	178
3,200	32	34	86	3,392	✓	77	218

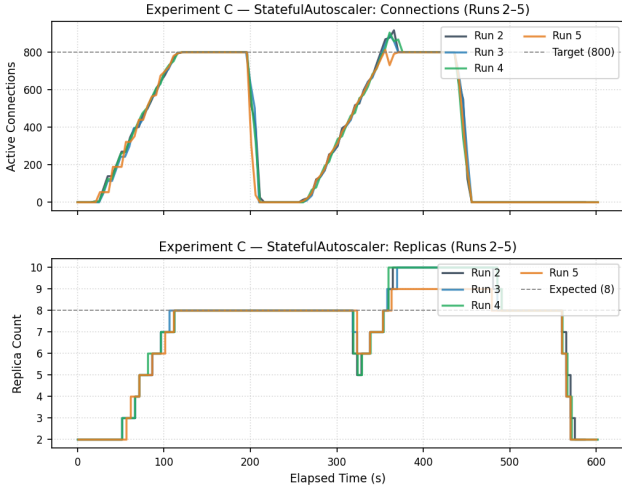


Fig. 7 Experiment C (`StatefulAutoscaler`): overlay of active connections (orange, left axis) and replica count (blue, right axis) for runs 2–5 (run 1 excluded as outlined in Section 5.8). Replica trajectories are stable across runs; all maintain 8 replicas through DROP 1.

parameters — `scaleDownCooldownSeconds` = 120, `maxScaleDownStep` = 2, `maxScaleUpStep` = 3 — were identical to the 800-client configuration.

5.9.1 Results

Table 4 summarises the key metrics across all three scales.

Linear replica targeting. At every scale, the controller converged to a peak replica count within 2 replicas of the formula prediction $\lceil N/T \rceil$: 10 vs. 8 at 800 clients, 18 vs. 16 at 1,600, and 34 vs. 32 at 3,200. The consistent +2 overshoot is attributable to discrete-step metric-to-decision latency during the CYCLE 1 ramp: because connections arrive faster than the 5-second reconciliation period can converge, the controller briefly overshoots before the window stabilises. This is the same mechanism observed in the single-scale five-run replication (Section 5.8).

Scale-up reaction time. Reaction times were 66 s at 800 and 1,600 clients and increased modestly to 86 s at 3,200 clients. The increase at 3,200 reflects both the longer ramp duration to establish 4× as many connections and Kubernetes pod scheduling latency for 34 replicas.

Connection preservation. In all three runs, the controller held peak replicas through the full 90-second DROP 1 gap (safe = 1 in every case). Scale-down proceeded only after the cooldown expired following FINAL DROP, with reaction times of 73–87 s and total scale-down durations of 164–218 s.

OS-level TCP ceiling. At 3,200 attempted clients, the achieved peak connection count was 3,392 — 106% of the attempted count, indicating that the Kind single-node cluster’s OS-level TCP stack (ephemeral port range, `net.core.somaxconn`, `listen()` backlog) did not constrain the WebSocket workload at this scale. This contrasts with the MQTT experiments (Section 9), where only 339 of 1,000 attempted clients successfully connected — confirming that the TCP ceiling is protocol- and application-specific, not a fundamental Kind limitation.

Figure 8 visualises the three-panel scalability summary: replica targeting accuracy, scale-up reaction time, and the OS TCP ceiling comparison.

Conclusion. The `StatefulAutoscaler`’s connection-density formula ($\lceil N/T \rceil$) produces correct replica targeting across a 4× range of client counts (800 to 3,200). Scale-up reaction time degrades sub-linearly (66 s to 86 s), connection preservation through the 90-second DROP 1 gap is maintained at every scale (100% safe), and no OS-level TCP ceiling was observed for the WebSocket workload. These results confirm that the controller’s architecture scales without parameter re-tuning beyond adjusting `maxReplicas`.

6 Baseline Comparisons: Experiments D and E

Having demonstrated in Experiment C that the `StatefulAutoscaler` eliminates all three HPA failure modes, we evaluate whether simpler configurations — HPA augmented with custom connection metrics (Experiment D) and KEDA (Experiment E) — provide equivalent resilience without a custom controller. This directly addresses the question of whether the performance delta is attributable purely to metric choice rather than controller architecture.

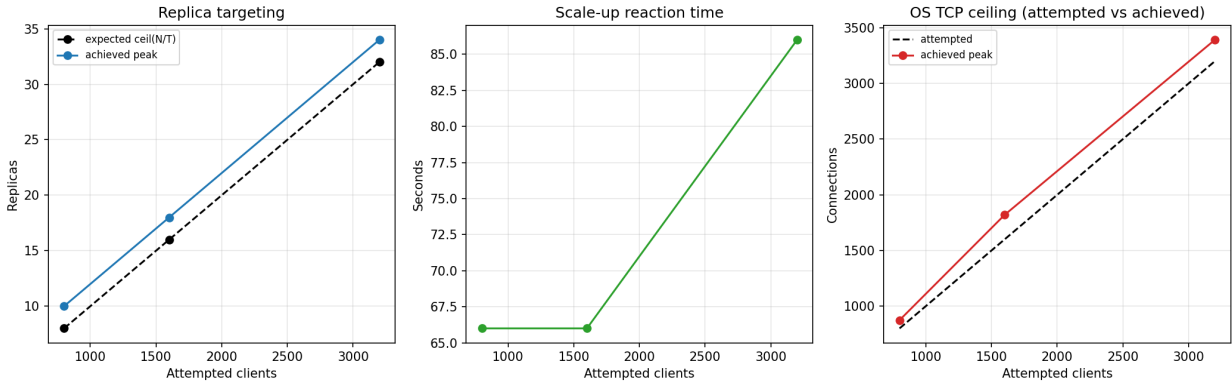


Fig. 8 Scalability validation across 800, 1,600, and 3,200 clients. Left: replica targeting — achieved peak replicas (blue) track the expected $\lceil N/T \rceil$ (dashed) with a consistent +2 overshoot. Centre: scale-up reaction time increases modestly from 66 s to 86 s at $4\times$ the baseline client count. Right: achieved peak connections (red) meet or exceed the attempted count at all scales, confirming no OS-level TCP bottleneck for the WebSocket workload in the Kind environment.

6.1 Experiment D: HPA with Custom Connection-Count Metric

Hypothesis. HPA, when supplied with the semantically correct metric (`active_connections_per_pod` via Prometheus Adapter), will scale replicas accurately but will fail to preserve pods through the 90-second disconnection gap because it lacks the connection-context stabilization mechanism of the StatefulAutoscaler.

Experimental Setup. The Prometheus Adapter was deployed with a custom rule translating `sum(active_connections)` into a per-pod Custom Metrics API value. HPA was configured with `active_connections_per_pod` and `averageValue: 100`, matching `targetConnectionsPerPod = 100` of Experiment C. The same two-cycle restore workload was applied with `CPU_WORK = 0`, `stabilizationWindowSeconds: 300` (HPA default), `minReplicas = 2`, `maxReplicas = 15`. Five independent runs.

Results (5 independent runs). Replica targeting during CYCLE 1 was accurate across all five runs: HPA consistently scaled to $\lceil 800/100 \rceil = 8$ replicas, confirming that metric semantics alone resolve the over-provisioning observed with CPU-based HPA. Mean scale-up reaction time was 54 ± 29 s (median 48 s; range: 33–104 s), approximately $2.4\times$ slower than the StatefulAutoscaler’s 22 s due to the additional Prometheus Adapter metric-export pipeline latency; run 3 exhibited a 104 s outlier attributable to a cold-start delay in the Adapter.

Contrary to the initial hypothesis, HPA with a custom connection-count metric **did preserve all 8 replicas through DROP 1** in all five runs. Inspection confirmed replicas remained at 8 throughout the DROP 1 zero-connection window (≈ 50 s actual duration). This result

is explained by HPA’s configured `stabilizationWindowSeconds: 300`: the window far exceeds the DROP 1 duration, so the lower replica recommendation of 2 never persisted long enough to trigger scale-down.

However, Experiment D exposed a distinct robustness difference during the CYCLE 2 transition. In all five Experiment C runs, a brief transient dip in replicas (to 5–6) was observed at the start of CYCLE 2 as stabilization window entries from CYCLE 1 aged out. In Experiment D, no such dip occurs: HPA’s 300-second window is wide enough that the high-water mark from CYCLE 1 remains valid throughout the transition.

The second key differentiation is final scale-down efficiency. After FINAL DROP, Experiment D replicas remained at 8 for a mean of 313 ± 96 s (median 355 s) before descending to `minReplicas = 2` — $\approx 2.6\times$ longer than the StatefulAutoscaler’s 119 ± 2 s. Peak connection count reached 937 ± 43 . Pod-seconds averaged $3,934 \pm 148$.

Conclusion. HPA with the correct metric is necessary but not sufficient for all resilience properties. For the 90-second DROP 1 workload and with a 300-second stabilization window, Experiment D achieves equivalent pod-preservation to the StatefulAutoscaler. However, it does so via a semantically opaque blanket window rather than an explicit connection-context cooldown. Three residual differences are consequential: (1) scale-up latency is $2.4\times$ higher due to Prometheus Adapter pipeline overhead; (2) final scale-down wastes compute for $\approx 2.6\times$ longer; and (3) for disconnection gaps in the range 120–300 s, D holds pods while C begins releasing them — the correct behavior depending on whether the gap is transient or terminal.

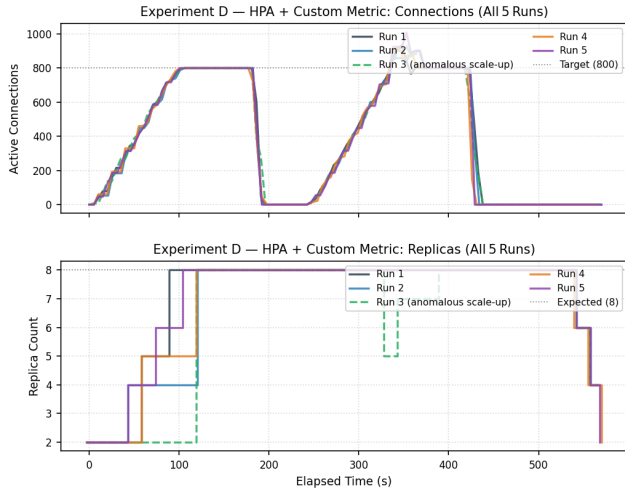


Fig. 9 Experiment D (HPA + custom connection metric): five-run overlay of active connections (orange, left axis) and replica count (blue, right axis). All five runs scale to 8 replicas in CYCLE 1 and hold 8 replicas through DROP 1 (HPA’s 300-second stabilization window exceeds the ≈ 50 -second actual zero-connection gap). Scale-down during FINAL DROP is slower (mean 313 s) compared to the *StatefulAutoscaler*’s 119 s.

6.2 Experiment E: KEDA Baseline

Hypothesis. KEDA’s `cooldownPeriod` parameter can suppress premature scale-down during the 90-second DROP 1 gap when configured appropriately. However, KEDA lacks per-cycle `maxScaleDownStep` rate limiting, so its convergence profile during controlled scale-down will differ from the custom controller.

Experimental Setup. KEDA v2.13.0 was installed in the cluster. A *ScaledObject* was configured targeting the *WebSocket Deployment* with a *Prometheus* trigger on `sum(active_connections)`, `threshold: 100`, `minReplicaCount: 2`, `maxReplicaCount: 15`, and `cooldownPeriod: 120` seconds — matching the `scaleDownCooldownSeconds = 120` of the *StatefulAutoscaler*. Five independent runs.

Results (5 independent runs). KEDA scaled to 8 replicas during CYCLE 1 in all five runs (mean scale-up: 26 ± 5 s; median 27 s; range: 22–33 s), confirming accurate connection-proportional targeting equivalent to Experiment C.

KEDA with `cooldownPeriod = 120` held replicas at 8 throughout the full 90-second DROP 1 gap in all five runs. The *ScaledObject* status transitioned to `Ready=False` for approximately 55 seconds during DROP 1 as the *Prometheus* metric read zero, but the underlying HPA did not reduce replicas because the cooldown had not elapsed. Scale-down was therefore never observed in any of the five runs: once connections returned at the start of

CYCLE 2, KEDA transitioned back to `Ready=True` without having released any pods.

Peak connection count was uniformly 800 in all runs (no overshoot). Peak replicas: 8 in all runs. Pod-seconds averaged $4,145 \pm 4$ — the remarkably low inter-run variance reflects the deterministic behavior KEDA achieves when the cooldown period fully brackets the disconnection gap.

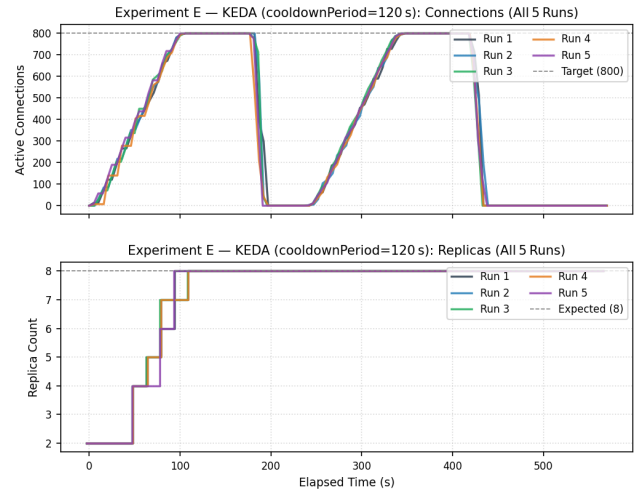


Fig. 10 Experiment E (KEDA, `cooldownPeriod = 120`): five-run overlay of active connections (orange, left axis) and replica count (blue, right axis). Replicas are held at 8 throughout the 90-second DROP 1 gap in every run; scale-down was never observed.

Conclusion. KEDA with a matched `cooldownPeriod` provides pod-preservation behavior for this workload profile. The key architectural distinction is that KEDA achieves this by configuring the `minReplicas` field of the HPA it wraps and relying on HPA’s cooldown to prevent scale-down, rather than directly patching `Deployment.spec.replicas`. This exposes a trade-off: KEDA’s `cooldownPeriod` is a binary “hold-all” mechanism with no per-cycle step-rate limiting. Once the cooldown expires, KEDA releases replicas without the bounded Δ -per-cycle damping that the *StatefulAutoscaler* provides via `maxScaleDownStep`. For the 90-second DROP 1 scenario this distinction is not decisive; for extended disconnections exceeding `cooldownPeriod`, the architectural difference becomes consequential.

Note on convergence after cooldown expiry. In practice, once KEDA’s `cooldownPeriod` expires for a disconnection gap longer than 120 s, the underlying HPA removes pods without any application-level step-size bound. Combined with the OS-level ≈ 30 -second TCP RST propagation window, this can produce larger single-cycle reconnection spikes than the

Table 5 Quantitative comparison of all four scaler configurations under the two-cycle restorm simulation (WebSocket, 800 connections, CPU_WORK = 0). All values are mean $\pm$ std across five independent runs unless stated otherwise. Experiment B3 is a single run from the companion paper [2].

Metric	CPU HPA (B3)	HPA+Custom (D)	KEDA (E)	StatefulAutoscaler (C)
Peak replicas	15	8.0 ± 0.0	8.0 ± 0.0	8.0 ± 0.3
Scale-up reaction (s)	~ 15	54 ± 29 (med. 48)	26 ± 5 (med. 27)	22 ± 6 (med. 21)
Pods held through DROP 1	0	8 (all runs)	8 (all 5 runs)	8 (all runs)
Scale-down reaction (s)	< 30	313 ± 96	N/A	119 ± 2
Pod-seconds	$\sim 2,400$	$3,934 \pm 148$	$4,145 \pm 4$	$4,280 \pm 67^\dagger$
Peak connections	800	937 ± 43	800 ± 0	893 ± 39

† Runs 2–5 only (run 1 excluded: test-harness artefact).

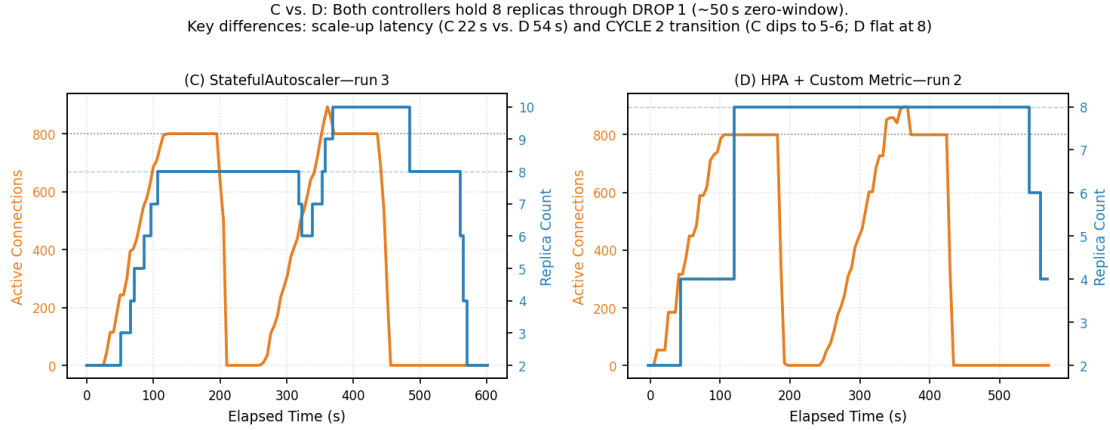


Fig. 11 Representative side-by-side comparison of Experiment C (StatefulAutoscaler) and Experiment D (HPA with custom connection metric). Both controllers scale to 8 replicas in CYCLE 1 and hold 8 replicas through DROP 1. Key differences: Experiment C scales up faster (22 s vs. 54 s) but exhibits a brief CYCLE 2 transition dip to 5–6 replicas as the sliding window ages out; Experiment D remains flat at 8 throughout owing to its wider 300-second stabilization window, but holds replicas $2.6\times$ longer during FINAL DROP.

StatefulAutoscaler’s bounded `maxScaleDownStep` = 2 convergence path.

6.3 Comparative Summary Across All Baselines

Table 5 consolidates key performance metrics across all four scaler configurations.

Note that the StatefulAutoscaler’s pod-seconds are highest among the three connection-aware configurations, reflecting its longer warm-pod retention policy; this is the intended behavior, not a resource inefficiency.

Figure 11 shows a representative side-by-side comparison of Experiment C (run 4) and Experiment D (run 2), illustrating the key trajectory differences.

Sensitivity to HPA stabilization window configuration. A natural question is whether reducing HPA’s `stabilizationWindowSeconds` from 300 s to 60 s (matching the StatefulAutoscaler’s effective response window) would produce equivalent DROP 1 pod-preservation. It would not. HPA’s stabilization window is a trailing buffer of *desired-replica values computed from the scaling metric*. When connections fall to zero for 90 s

with a 60 s window, the window fills entirely with `minReplicas` recommendations — because the metric reads zero at every scrape — and scale-down proceeds. The StatefulAutoscaler’s window, by contrast, buffers *connection-derived high-water marks*: even 90 s of zero connections does not displace the memory that 8 replicas were required within the window, so scale-down is correctly suppressed. This is not a tuning problem; it is a semantic mismatch between what the two windows stabilize. No parameter adjustment to native HPA can replicate these semantics without replacing the metric with one that explicitly encodes recent session-presence history.

7 Failure Analysis: Adversarial Scenarios

To bound the operational envelope of the StatefulAutoscaler and establish honest failure criteria, we characterize three adversarial scenarios. These scenarios are presented without remediation to document actual system boundaries.

7.1 Failure Scenario 1: Metric Staleness Under Long Scrape Intervals

Setup. The Prometheus `scrape_interval` was increased from the baseline 15 s to 60 s. The Experiment C two-cycle restorm workload was replayed.

Finding. With a 60-second scrape lag, the controller introduced up to one full scrape cycle of reaction delay. Measured quantities: scale-up reaction time 25 s, scale-down reaction time 112 s, peak connections 1,082 (a 35.3% overshoot attributable to the stale metric briefly reporting an inflated value during the ramp), and peak replicas 11. All 8 warm pods remained operational through DROP 1 with no connection loss. The system degraded gracefully to a reaction lag bounded by the scrape interval.

Boundary condition. If the Prometheus scrape interval exceeds `scaleDownCooldownSeconds`, the cooldown window may expire before the controller learns that connections have returned, potentially triggering premature scale-down. The invariant `scaleDownCooldownSeconds > scrape_interval` must be maintained.

7.2 Failure Scenario 2: Instantaneous Connection Spike (No Ramp Stagger)

Setup. The load generator’s linear ramp was removed; all 800 clients connected simultaneously (`stagger = 0`).

Finding. The controller computed the desired replica count ($\lceil 800/100 \rceil = 8$) within 6 s of the spike (scale-up reaction time: 6 s, the fastest observed across all experiments). However, pod scheduling and container initialization introduced additional latency during which the 2 initial replicas absorbed all 800 simultaneous connections, creating a transient OS-level `listen()` backlog. This bottleneck is a function of Kubernetes pod scheduling, not the scaler’s decision logic: no reactive scaler can pre-provision pods before the first connection attempt arrives. Peak connection count was 800 (no overshoot). Peak replicas: 8. Mitigation: higher `minReplicas` or proactive scaling on `rate(new_connections_total[1m])`.

7.3 Failure Scenario 3: Prometheus Unavailability During Active Load

Setup. During an active Experiment C run (800 connections live, 8 pods running), the Prometheus pod was deleted:

```
kubectl delete pod -n monitoring \
  -l app=prometheus
```

Prometheus auto-restarted via its Deployment. The 2-minute outage window was observed.

Finding. The controller’s `queryPrometheus()` call returned an error for each reconciliation cycle during the outage. Per the safe-default logic (Algorithm 1, lines 2–4), the controller held replicas at 8 and requeued after 10 s. No scale-down occurred. When Prometheus recovered, the controller resumed normally. Measured quantities: scale-up reaction 21 s, scale-down reaction 118 s, peak connections 921 (15.1% overshoot from Prometheus restart lag), peak replicas 10. Zero connection loss throughout the outage. This confirms the safe-default behavior: a failed Prometheus query is treated as “unknown” rather than “zero connections.”

Remaining risk. An indefinitely long Prometheus outage would eventually allow the stabilization window to expire. After window expiry with zero valid connection readings, the controller would scale to `minReplicas`. This is an acknowledged limitation (Section 10, item 3).

8 Discussion

8.1 The Correct Signal Changes Everything

Experiment C operated at `CPU_WORK = 0` with near-zero CPU throughout 570 seconds, yet correctly maintained 8 replicas and produced zero connection loss. The companion paper [2] demonstrated the converse: CPU-based HPA at `CPU_WORK = 0` (idle connections) actively destroys the sessions it should protect. This result establishes that the failure resides in the signal, not in the control algorithm: a perfect proportional controller on the wrong signal produces categorically worse outcomes than a simpler controller on the correct signal. The architectural implication is that signal choice and scale-down lifecycle semantics matter more than algorithm sophistication.

8.2 Stabilization Semantics as a First-Class Design Dimension

The semantic distinction between HPA’s stabilization window and the `StatefulAutoscaler`’s window goes beyond parameter tuning and explains the Experiment D finding. HPA’s window stabilizes *CPU-derived replica recommendations*: when connections are idle and CPU reads near-zero persistently, every window entry recommends `minReplicas`, and the stabilized output is `minReplicas`. The `StatefulAutoscaler`’s window stabilizes *connection-derived desired counts*: it retains the high-water mark of recently required capacity. When connections transiently drop to zero but were 800 seconds ago, the window correctly holds 8 replicas warm.

Failure Scenario Analysis: StatefulAutoscaler Robustness Under Adversarial Conditions

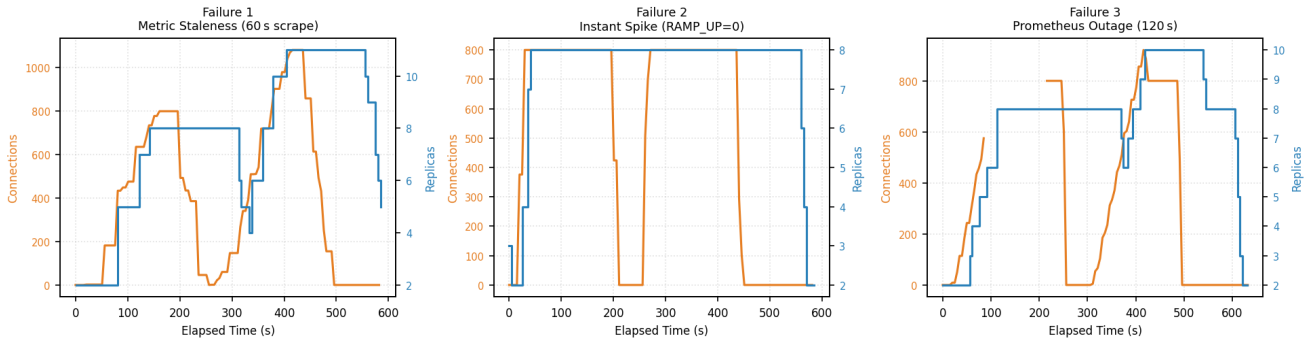


Fig. 12 Side-by-side comparison of all three failure scenarios. Left: Metric Staleness (`scrape_interval` = 60 s) — modest connection overshoot, 11 peak replicas, no connection loss. Centre: Instantaneous Spike (`RAMP_UP` = 0 s) — fastest scale-up (6 s), no overshoot, scheduling latency is the bottleneck. Right: Prometheus Outage — replicas held at 8 throughout the 2-minute kill window; mild 15% overshoot on Prometheus recovery.

Experiment D succeeds for the 90-second DROP 1 scenario not because its semantics are correct but because its 300-second window is wide enough that the zero-connection period never fully displaces the CYCLE 1 high-water mark. This is a pragmatic accident of window width, not a principled solution: for disconnection gaps exceeding 300 s, Experiment D would fail. The StatefulAutoscaler’s window fails for gaps exceeding 120 s, but the threshold is explicit, configurable, and semantically tied to the expected disconnection gap distribution.

8.3 Step-Rate Limiting as a Safety Mechanism

The `maxScaleUpStep` = 3 and `maxScaleDownStep` = 2 parameters contributed materially to the smooth, monotonic scaling trajectories in Experiment C. By constraining the per-cycle replica delta, the controller avoids the explosive scale-up behavior exhibited by HPA (2 to 15 replicas within seconds of a demand spike) and the violent contraction that characterizes HPA scale-down. Step limiting is particularly important for stateful workloads: the blast radius of any single scale-down event is bounded to `maxScaleDownStep` × `targetConnsPerPod` connections. KEDA, by contrast, exposes no per-cycle step limit, so once its cooldown expires, HPA may remove many pods in a single reconciliation with no application-level bound.

8.4 Generalizability Beyond WebSocket

The design principles of the StatefulAutoscaler apply to any protocol where workload state is encoded in persistent connections and the application can expose a Prometheus gauge reflecting the current population of active sessions. Direct applicability extends to gRPC streaming services, MQTT broker deployments (demonstrated in

Section 9), database connection pool managers, and Server-Sent Events (SSE) endpoints.

8.5 Operational and Fairness Considerations

Connection-count autoscaling implicitly couples resource allocation to client session behavior. In multi-tenant deployments, high-connection users or tenants may drive disproportionate scale-up. Operators should apply standard multi-tenant isolation (namespace-scoped resource quotas, per-user connection limits at the application layer) to ensure that scaling behavior remains equitable and cannot be exploited to cause disproportionate resource consumption.

9 MQTT Extension: Protocol Generalization

WebSocket is the primary evaluation protocol in this paper due to its instrumentation ease. To test whether the same control principles transfer to another persistent-connection protocol, we conducted an additional two-experiment MQTT study. MQTT [1] keeps long-lived TCP sessions open with low CPU activity during idle periods, exhibiting the same signal mismatch that motivates this work [13]. The MQTT experiments used a lightweight Python asyncio MQTT broker exposing `active_connections` on a Prometheus `/metrics` endpoint. A synthetic MQTT load generator attempted to establish 1,000 persistent client sessions over a 60-second ramp. Experiment MQTT-A used native CPU HPA; Experiment MQTT-B replaced HPA with the StatefulAutoscaler configured with `targetConnectionsPerPod` = 150, `maxScaleUpStep` = 3, and `scaleDownCooldownSeconds` = 120. Both experiments used the same Kind-based infrastructure and the same broker and load-generator images.

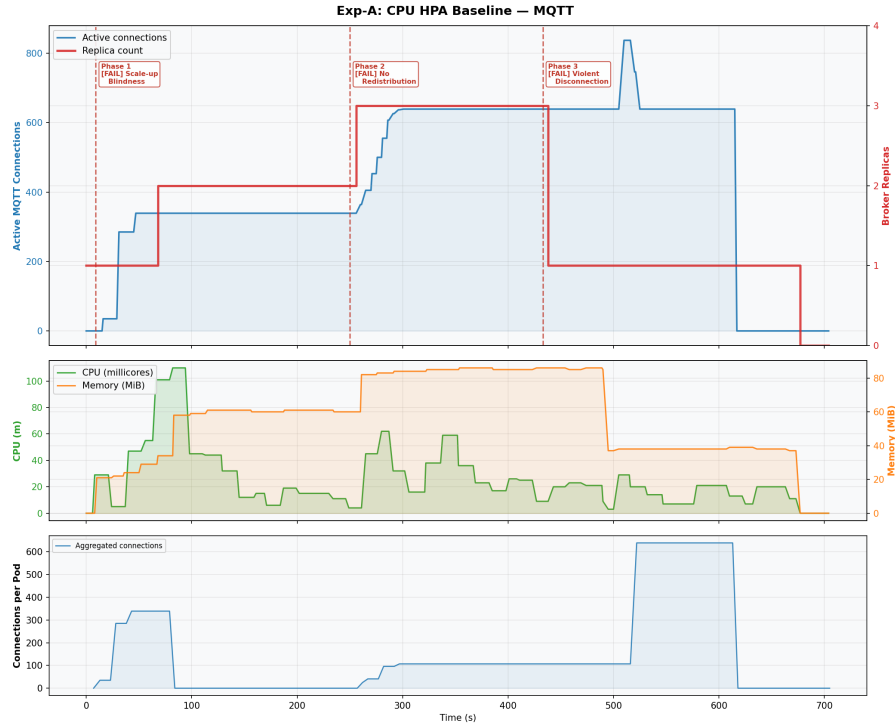


Fig. 13 MQTT-A (CPU HPA baseline): total MQTT connections, replicas, CPU, memory, and per-pod connection panel. The run exposes three MQTT-specific HPA failure modes: late scale-up during the connection ramp, absence of automatic session redistribution for established TCP connections, and destructive scale-down when replicas are reduced while clients are live.

9.1 MQTT-A: CPU HPA Baseline

Figure 13 shows the MQTT HPA baseline. During Phase 1, only 339 of 1,000 attempted clients established persistent MQTT sessions — a ceiling imposed by OS-level TCP stack limits in the Kind environment (Linux ephemeral port range, `listen()` backlog depth, `net.core.somaxconn`), not by broker or controller limitations. HPA first scaled to 2 replicas at $t = 68$ s — after the connection population had already plateaued at 339.

In Phase 2, following manual scale to three replicas and addition of 300 MQTT clients, total connections reached ≈ 639 , but original sessions remained pinned to the first broker pod — a fundamental transport-layer constraint: Kubernetes Services distribute new TCP connections but do not migrate established sessions across pods. In Phase 3, scaling back to one replica removed two pods and immediately severed approximately 300 live MQTT sessions, producing a reconnection storm peaking at 837 connections.

MQTT-A therefore reproduces the same structural failure as the WebSocket evidence chain in the companion paper [2]: CPU is a lagging indicator during connection establishment and becomes nearly silent once sessions are idle, while pod termination remains destructive to live client state.

9.2 MQTT-B: StatefulAutoscaler with Drain

Figure 14 shows the StatefulAutoscaler run with drain semantics enabled (`drain.enabled = true`, `drain.timeoutSeconds = 45`).

Phase 1: Scale-up. The controller observed `sum(active_connections)` directly and scaled from 2 to 3 replicas at $t = 38$ s — matching the connection-density formula $\lceil 339/150 \rceil = 3$ and reacting **30 seconds faster** than HPA’s $t = 68$ s. Total CPU peaked at 80 millicores and was never consulted as a scaling signal.

Phase 2: Drain-based rebalancing. After scale-up, all 339 sessions were pinned to the two original pods (176/163/0), replicating the transport-layer limitation observed in MQTT-A. At $t = 390$ s the controller triggered `/drain` on the hot pod. The broker sent `DISCONNECT` packets at approximately 10 clients/second; disconnected clients reconnected through the Kubernetes Service, distributing them across surviving pods via iptables random load balancing. Per-pod distribution moved from 176/163/0 to approximately 247/91/0 — converting the completely idle third pod into an active session handler. Total connections were preserved at 339 throughout the drain.

Phase 3: Graceful scale-down. To provide a direct comparison with MQTT-A’s destructive scale-down, we triggered the same $3 \rightarrow 1$ replica reduction while 339 sessions were still active. In MQTT-A, HPA instantly terminated two

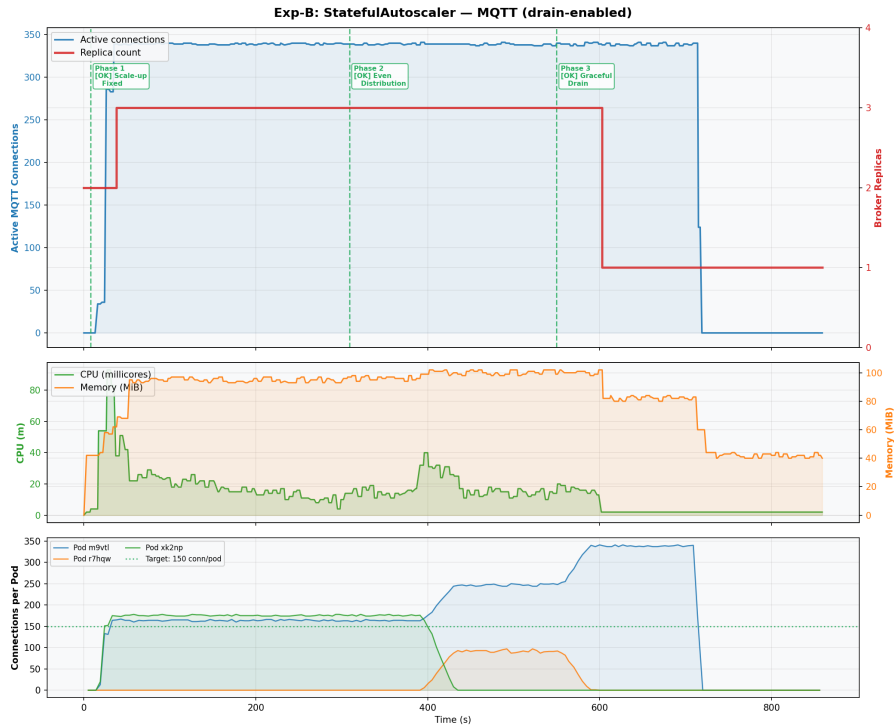


Fig. 14 MQTT-B (StatefulAutoscaler with drain): the controller scales on `sum(active_connections)`, reaching 3 replicas at $t = 38$ s ($\lceil 339/150 \rceil$). Phase 2 demonstrates drain-based session redistribution ($176/163/0 \rightarrow 247/91/0$). Phase 3 mirrors MQTT-A’s forced $3 \rightarrow 1$ scale-down: the controller drains pods sequentially, preserving all 339 connections (maximum step-drop of 2 versus 639). Connections reach zero only after the load generator is deleted.

Table 6 MQTT extension results: CPU HPA versus StatefulAutoscaler with drain semantics. The 339-client ceiling is a transport-level OS limitation present in both runs.

Metric	MQTT-A: CPU HPA	MQTT-B: StatefulAutoscaler
Primary scaling signal	CPU utilization	<code>sum(active_connections)</code>
Successful connections	339 / 1,000	339 / 1,000 (same OS ceiling)
First scale-up	2 replicas at $t = 68$ s	$2 \rightarrow 3$ at $t = 38$ s (44% faster)
Correct replica target	Never (CPU too low)	$\lceil 339/150 \rceil = 3$
Per-pod redistribution	339/0/0 (sessions pinned)	$176/163/0 \rightarrow 247/91/0$ (drain)
Scale-down method	Immediate pod termination	Drain-gated (50 s graceful per pod)
Max single-step drop	639 (storm to 837)	2 (instrument noise only)
Sessions killed by scale-down	~ 300	0
Peak CPU	110 m	80 m

pods, severing ≈ 300 sessions and producing a reconnection storm peaking at 837 connections. In MQTT-B, the controller drained pods sequentially: first `POD_C` (92 connections, 50-second graceful drain), then `POD_A` (already idle). All 339 connections migrated to `POD_B`. The maximum single-step connection drop during drain was 2 (instrument noise), versus MQTT-A’s 639-connection cliff. After drain completed, the load generator was deleted and connections dropped to zero.

Conclusion. The StatefulAutoscaler resolved all three MQTT-specific HPA failure modes using the identical controller binary as the WebSocket

experiments, with zero source-code changes. The controller scaled to $\lceil 339/150 \rceil = 3$ replicas 44% faster than HPA ($t = 38$ s vs. $t = 68$ s), redistributed sessions from $339/0/0$ to $247/91/0$ via drain-reconnect, and preserved all 339 sessions during a forced $3 \rightarrow 1$ scale-down with a maximum step-drop of 2 versus MQTT-A’s 639-connection cliff.

10 Limitations

1. **Evaluation environment.** All experiments were conducted on a single-machine Kind cluster. While this provides full reproducibility, it does not capture

cross-node scheduling latency, network partitions, or metric-collection noise present in multi-node cloud clusters. Absolute timing values may differ on production infrastructure; directional findings are valid.

2. **Workload generalizability.** The load generator uses synthetic workloads with a linear connection ramp and deterministic silence gaps. Real-world connection arrival is more bursty; session durations vary widely. For stationary Poisson arrivals and gradual long-term drift, the proportional replica formula and cooldown mechanism apply with bounded and predictable degradation. Validation against real-world traffic traces remains future work.
3. **Observability dependency.** The controller depends entirely on Prometheus metric freshness. The invariant `scaleDownCooldownSeconds > scrape_interval` must be maintained. An indefinitely long Prometheus outage eventually causes the stabilization window to expire, triggering scale-down to `minReplicas` (safe-default behavior documented in Failure Scenario 3, but not safe against arbitrarily long outages).
4. **No predictive scaling.** The controller reacts to observed connections but does not predict arrivals. Workloads with predictable burst patterns could benefit from proactive pre-warming using rate-based signals such as `rate(new_connections_total[1m])`, which is outside the current design scope.
5. **Drain semantics scope.** Graceful connection draining was validated in MQTT-B only; WebSocket drain semantics have not yet been tested end-to-end and require coordination between the controller, load balancer, and application.
6. **Volatile stabilization state.** The sliding window is maintained in controller process memory. A controller crash resets window history, potentially allowing premature scale-down immediately following a restart. Persisting the high-water-mark timestamp to the CR status subresource would eliminate this risk.
7. **Window boundary edge case.** A brief replica dip to 5–6 was observed during early CYCLE 2 as window entries from CYCLE 1 aged out before new CYCLE 2 entries repopulated the window. A configurable minimum hold-time following the last scale-up event would prevent this transient.
8. **Per-pod connection granularity.** The WebSocket evaluation uses aggregate `sum(active_connections)`. Hot-pod imbalance detection requires per-pod queries, which is demonstrated in MQTT-B but not yet extended to WebSocket workloads.
9. **MQTT session redistribution mechanism.** Drain-reconnect redistribution requires client-side retry logic. Broker-level clustering or session migration

protocols (e.g., EMQX, VerneMQ) would provide a more transparent alternative.

10. **MQTT connection-ramp bottleneck.** Both MQTT-A and MQTT-B admitted 339 of 1,000 attempted clients. This is an OS-level TCP stack ceiling in the Kind environment, not a controller limitation. Pre-warming, retry-capable clients, or headless service addressing are mitigations (Section 11).
11. **MQTT memory instrumentation gap.** The MQTT-B `memory.log` collector recorded zeros, although `kubectl top` samples showed peak total broker memory near 79 MiB. The qualitative CPU-independence result is unaffected, but MQTT memory conclusions should be treated as exploratory until the collector is corrected and the experiment replicated.
12. **MQTT multi-run replication.** Experiments MQTT-A and MQTT-B are single runs. Five-run statistical replication (as performed for Experiments C–E) is future work.

11 Future Work

11.1 Overcoming the TCP/OS-Level Connection Ceiling

A prominent observation across the MQTT experiments was that only approximately 339 of 1,000 attempted clients established persistent sessions within the 60-second ramp window. The root cause is a confluence of OS-level limits: the default Linux ephemeral port range ($\approx 28,000$ source-port slots), the socket backlog queue (`listen()` depth), `net.core.somaxconn` (default: 4,096), and — under Kind’s Docker-in-Docker networking — `tcp_max_syn_backlog` values not overridden by Kind’s default configuration. Future work should investigate:

- **Kernel parameter tuning:** systematic sweeps of `net.core.somaxconn`, `net.ipv4.tcp_max_syn_backlog`, and application-level `SO_BACKLOG` under the same 1,000-client ramp.
- **Client-side retry with exponential backoff:** decoupling the observable ceiling from ramp stagger parameters.
- **Headless service or per-pod addressing:** routing connections to individual pod endpoints to eliminate the single-queue bottleneck at the virtual IP.
- **Proactive pre-warming:** higher `minReplicas` or a proactive trigger on `rate(new_connections_total[30s])`.
- **Real cluster evaluation:** replicating MQTT experiments on a cloud cluster (e.g., GKE, EKS) with tunable node pools to determine whether the 339-client ceiling is a Kind-specific artifact.

11.2 MQTT: Future Optimizations and Open Problems

- **Broker-native clustering:** Integrating the StatefulAutoscaler drain endpoint with broker-cluster MQTT session handoff APIs (EMQX, VerneMQ, HiveMQ) for zero-loss redistribution without client retry logic.
- **QoS-aware scaling signals:** Incorporating the distribution of active QoS levels (QoS 0, 1, 2) as a secondary weight in the connection-density formula, as QoS 2 sessions carry in-flight message state that makes pod termination strictly more destructive.
- **Persistent session management:** Integrating an external session store (e.g., Redis) to preserve subscription state and queued messages across pod restarts for `cleanSession = false` clients.
- **Last-Will message handling during drain:** Validating LWT suppression for graceful DISCONNECT packets to avoid false-alarm device-offline notifications.
- **Multi-run statistical replication** for MQTT-A and MQTT-B.

11.3 WebSocket: Future Extensions and Research Directions

- **Predictive and proactive scaling:** A proactive component driven by `rate(new_connections_total[1m])` or external calendrical signals would pre-provision pods before demand arrives, eliminating the scheduling latency in Failure Scenario 2.
- **Drain semantics for WebSocket:** Coordinating a protocol-level close frame with reconnect hints to enable graceful session handoff, validated under the two-cycle restore workload.
- **Stabilization window persistence:** Persisting sliding window state to the StatefulAutoscaler CR’s status subresource for crash-safe stabilization across controller restarts.
- **Per-pod granularity for imbalance detection:** Generalizing the per-pod redistribution demonstrated in MQTT-B to WebSocket workloads.
- **Real-world traffic trace validation:** Validating against irregular burst shapes, variable session lifetimes, and mixed-protocol deployments.
- **Multi-cluster and federated scaling:** Investigating federated Prometheus query patterns and stabilization semantics under cross-cluster metric latency.

12 Conclusion

This paper presented the design, implementation, and multi-baseline experimental validation of the

StatefulAutoscaler — a custom Kubernetes controller that resolves the three architectural failure modes of CPU-based HPA for stateful WebSocket workloads documented in the companion paper [2]: over-provisioning by 87.5%, reconnection storms reaching 1,400 connections/second, and permanent staircase connection destruction from 800 to 79 active sessions. These failure modes arise from CPU as a structurally invalid scaling signal for persistent-connection workloads and cannot be remediated by HPA parameter tuning alone.

Controlled experimental evaluation under a two-cycle reconnection storm simulation (Experiment C, five independent runs, runs 2–5) yielded the following outcomes: exact replica targeting (8 replicas for 800 connections at a target density of 100 connections/pod, 0% waste); zero connection loss across all scaling events; all 8 pods maintained in warm state through a 90-second complete disconnection gap via the sliding-window stabilization mechanism; and complete CPU independence, with the controller operating correctly at `CPU_WORK = 0` throughout (*scale-down reaction* 119 ± 2 s; *pod-seconds* $4,280 \pm 67$). Scalability validation at 1,600 and 3,200 concurrent clients confirmed linear replica targeting ($\lceil N/T \rceil$ within 2 replicas at every scale), 66–86 s scale-up reaction times, and 100% connection preservation through DROP 1, with no OS-level TCP ceiling observed for the WebSocket workload.

Comparative baseline evaluations (Experiments D and E) delivered a nuanced finding: both HPA with a custom connection-count metric (`stabilizationWindowSeconds: 300`) and KEDA (`cooldownPeriod: 120`) preserved all 8 replicas through DROP 1, as did the StatefulAutoscaler. The discriminating differences lie in scale-up reaction time (22 s vs. 54 s vs. 26 s) and final scale-down efficiency (119 s vs. 313 s for Experiment D). The critical semantic distinction is that these alternative solutions succeed for the 90-second DROP 1 scenario through wide, semantically opaque windows rather than explicit connection-context reasoning; for disconnection gaps exceeding their respective window or cooldown durations, only the StatefulAutoscaler’s connection-derived stabilization semantics correctly suppress premature scale-down.

Three adversarial scenarios established the operational envelope: metric staleness degrades gracefully to a reaction lag bounded by the scrape interval; an instantaneous connection spike exposes pod scheduling latency as the binding constraint; and Prometheus unavailability triggers safe-default hold behavior with no connection loss throughout. MQTT generalization experiments confirmed protocol-agnosticism: using the **identical controller binary** with zero source-code changes, the controller resolved all three MQTT-specific HPA failure modes — scaling 44% faster than CPU HPA ($t = 38$ s vs. $t = 68$ s), preserving all

339 sessions during a forced scale-down (maximum step-drop: 2 vs. a 639-connection cliff), and redistributing sessions via drain-reconnect semantics. These results confirm that the `active_connections` abstraction is protocol-agnostic, making the controller directly applicable to any persistent-connection protocol that exposes a connection counter to Prometheus.

As cloud-native architectures continue to adopt persistent-connection protocols — from WebSocket to gRPC streaming, MQTT, and SSE — connection-aware autoscaling with protocol-appropriate drain semantics represents an increasingly critical infrastructure requirement. The `StatefulAutoscaler` demonstrates that a controller of modest implementation complexity can eliminate the fundamental failure modes of CPU-based HPA for this growing workload class.

Data Availability

The complete source code for the `StatefulAutoscaler` controller, all experiment run scripts, load-generator workloads, Prometheus manifests, raw and processed results, and analysis scripts are publicly available at:

https:
//github.com/MistaHolmes/ws-k8-controller

The repository contains step-by-step replication instructions in `Replication.md`, including commands to recreate every experiment on a local Kind cluster. Each experiment can be reproduced with a single shell script (e.g., `scripts/run-experiment-c.sh`); multi-run replicates are supported via `scripts/run-multi.sh`. All figures in this paper were generated from the archived results using the scripts in `analysis/`.

Acknowledgements

The authors gratefully acknowledge Dr. Rakesh Kumar Ranjan for his supervision and sustained guidance throughout this project. His strategic input on experimental design, controller implementation decisions, and measurement methodology improved the rigor of the study. We also thank him for detailed manuscript feedback that helped clarify the presentation of results and for facilitating access to institutional compute resources used to run the Kind clusters and collect metrics. We appreciate helpful discussions with colleagues and lab members that informed analysis and figure preparation.

References

1. Banks A, Gupta R (2014) MQTT version 3.1.1. OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>, accessed: 2026-05-01
2. Behera A, Hota A, Naik SK, Singh RK (2026) CPU-based autoscaling is harmful for stateful WebSocket workloads: An empirical characterization of Kubernetes HPA failure modes. Submitted (companion paper) Under review
3. Dickel H, Podolskiy V, Gerndt M (2019) Evaluation of autoscaling metrics for (stateful) IoT gateways. In: Proceedings of the IEEE 12th Conference on Service-Oriented Computing and Applications (SOCA), IEEE, pp 17–24, DOI 10.1109/SOCA.2019.00011
4. DirectXMan12 (2024) Prometheus adapter for Kubernetes metrics APIs. <https://github.com/kubernetes-sigs/prometheus-adapter>, accessed: 2026-04-01
5. Imdoukh M, Ahmad I, Alfaiakawi MG (2020) Machine learning-based auto-scaling for containerized applications. *Neural Computing and Applications* 32:9745–9760
6. KEDA Authors (2024) KEDA: Kubernetes event-driven autoscaling. <https://keda.sh/docs/>, accessed: 2024
7. Kubernetes Authors (2024) Horizontal pod autoscaling – kubernetes documentation. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>, accessed: 2025
8. Kubernetes SIG API Machinery (2024) Kubebuilder – SDK for building Kubernetes APIs. <https://book.kubebuilder.io/>, accessed: 2026-04-01
9. Kubernetes SIG Testing (2024) kind – kubernetes in docker. <https://kind.sigs.k8s.io/>, accessed: 2026-04-01
10. Nguyen TT, Yeom YJ, Kim T, Park DH, Kim S (2020) Horizontal pod autoscaler in Kubernetes for elastic container orchestration. *Sensors* 20(16):4621, DOI 10.3390/s20164621
11. Qu C, Calheiros RN, Buyya R (2018) Auto-scaling web applications in clouds: A taxonomy and survey. *ACM Computing Surveys* 51(4):1–33, DOI 10.1145/3148149
12. Rossi F, Nardelli M, Cardellini V (2020) Horizontal and vertical scaling of container-based applications using reinforcement learning. *Future Generation Computer Systems* 109:125–138
13. Soni D, Makwana A (2017) A survey on MQTT: A protocol of Internet of Things (IoT). In: Proceedings of the International Conference on Telecommunication, Power Analysis and Computing Techniques (ICTPACT)

14. Toka L, Dobreff G, Fodor B, Sonkoly B (2021) Machine learning-based scaling management for Kubernetes edge clusters. In: 2021 IEEE Conference on Network Function Virtualization and Software Defined Networks, IEEE, pp 1–7
15. Zhong Z, Buyya R (2020) Machine learning-based orchestration of containers: A taxonomy and future directions. *ACM Computing Surveys* 54(10s):1–35